

SSFT: Algorithm and Hardware Co-design for Structured Sparse Fine-Tuning of Large Language Models

Miao Yu
National University of Singapore
miao.yu@u.nus.edu

Trevor E. Carlson
National University of Singapore
tcarlson@comp.nus.edu.sg

Abstract—A significant number of users depend on Large Language Models (LLMs) for downstream tasks, but training LLMs from scratch remains prohibitively expensive. Sparse fine-tuning (SFT) has emerged as an effective strategy to reduce both the time and memory requirements of fine-tuning LLMs, achieving accuracy on par with fully fine-tuned models. Although SFT has the potential to achieve superior performance by minimizing computational requirements, SFT on GPUs often underperforms compared to dense algorithms like LoRA due to sparse data accesses that modern GPUs cannot efficiently handle. To address these issues, we propose Structured Sparse Fine-Tuning (SSFT). It comprises a novel algorithm, SSFT-Alg, which introduces predictable sparsity patterns to reduce memory access overhead and enhance regularity in the SFT process. To support SSFT-Alg, an accelerator, SSFT-Hw, is proposed to optimize SSFT-Alg through an innovative sparsity-aware design, avoiding the overhead of sparsity operations on GPUs and optimizing latency and energy efficiency. Experiments with relevant models and benchmarks demonstrate that SSFT achieves comparable accuracy to state-of-the-art models on BERT, LLaMA 2 7B, and LLaMA 2 13B. Moreover, SSFT-Hw outperforms both GPUs and the state-of-the-art sparsity-aware transformer accelerators in throughput by $51.0\times$ and $1.32\times$, respectively, while improving energy efficiency by $19.0\times$ and $1.48\times$.

I. INTRODUCTION

The scale of Large Language Models (LLMs), such as BERT [8], GPT-4 [1], and LLaMA [22], is fundamental to their cutting-edge performance. Adopting these models on downstream tasks is a universal demand, but the training cost makes it inaccessible for most users. Fine-tuning is the most practical approach for general users and companies to improve the accuracy of LLMs for these downstream tasks. However, fully fine-tuning these large models is hindered by their extensive memory and time requirements. Recent advancements in fine-tuning methods, particularly those employing Sparse Fine-Tuning (SFT) strategies, have demonstrated significant promise in reducing the memory requirements and latency while retaining accuracy [3], [4], [21]. These methods improve the balance between a minimal number of parameters and superior fine-tuning accuracy without introducing additional parameters into the LLM architectures like LoRA [11]. This approach effectively eliminates extra memory and computation overhead in practical applications.

Although SFT algorithms exhibit significant promise to minimize the number of parameter updates compared with

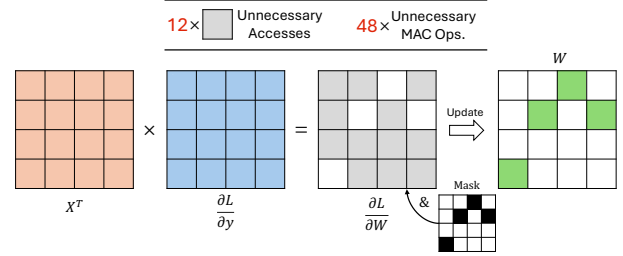


Fig. 1. An example of weight gradient generation and weight update stages with the previous sparse fine-tuning algorithm. While all weight gradients are calculated, only 4 are utilized to update weights, leading to 12 unnecessary accesses and 48 unnecessary MAC operations. SSFT can eliminate unnecessary accesses and operations.

dense fine-tuning techniques [12], their unique sparsity characteristics pose compatibility issues with contemporary processors, such as GPUs and Transformer accelerators, due to the irregularity in gradient calculation, the most time- and energy-intensive aspect of LLM fine-tuning. Specifically, as shown in Fig. 1, SFT algorithms either dynamically or statically calculate masks from the information derived from previous training steps. These masks are then applied to the gradients to update the current parameters. According to the principle of SFT algorithms, the bulk of gradients do not contribute to weight updates, leading to a large number of unnecessary memory accesses and MAC operations, given that a substantial majority of parameters ($> 95\%$) are frozen. Therefore, efficiently managing the masked gradients is key to improving the performance of SFT. However, neither the GPUs nor the state-of-the-art sparsity-aware transformer accelerators can efficiently manage the irregularity and high sparsity of masked gradients. As depicted in Fig. 1, computing the gradients matrix $\frac{\partial L}{\partial W}$ for parameter matrix W requires 16 memory accesses and 64 MAC operations but 12 accesses and 48 MAC operations are unnecessary for this algorithm, resulting in inefficiencies in both latency and energy consumption.

The design of current accelerators, particularly those for Transformers (the foundational architecture of LLMs), fails to efficiently support sparsity throughout fine-tuning [20], [23], [24], [28]. The state-of-the-art sparsity-aware transformer accelerator, TransCODE [24], demonstrates limitations in handling LLM fine-tuning with high sparsity because of the

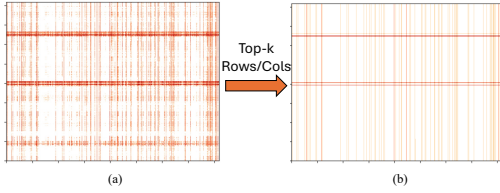


Fig. 2. (a) The distribution of top-k gradients in the parameters of Attention-Q in the first layer of LLaMA 2 7B. Similar structures are observed in crucial parameter matrices across Transformer-based LLMs like BERT and LLaMA. (b) SSFT selects the top-k rows and columns of crucial parameters to generate the structured mask of Attention-Q.

high energy cost and latency of processing irregular sparse data and the requirement to load all weights into the on-chip buffer for computation. Even though the design has proven its effectiveness for mid-size language models such as BERT_{TINY} [25] (~4M params) and BERT_{BASE} [8] (~110M params), it still struggles with larger size models such as BERT_{LARGE} [8] (~340M params) or LLaMA 2 [22] (> 7B params) due to the high memory requirements. Other designs, such as accelerators for SpM*SpM [19], fail to accelerate Transformer-based models adequately due to their inability to perform specific Transformer operations like Softmax and layer normalization [20].

A key insight is that *previous work has not been able to take advantage of structured information in SFT*, given that structured sparsity is recognized for its hardware compatibility in AI workloads [5], [6], [10]. To bridge the gap between SFT algorithms and hardware, we present Structured Sparse Fine-Tuning (SSFT), a collaborative approach at the algorithm (SSFT-Alg) and hardware levels (SSFT-Hw). Fig. 2(a) illustrates the distribution of original top-k parameters based on gradients, revealing row and column patterns that previous work has overlooked. Building on this observation, SSFT-Alg structurally selects rows and columns containing a top-k number of values, shown in Fig. 2(b), while maintaining comparable accuracy. This is because structured sparsity retains most of the essential information from the original crucial parameters. Meanwhile, given that structured sparsity is recognized for its hardware compatibility in AI workloads [5], [6], [10], attributable to uniform memory access patterns and computation, it gives SSFT-Alg the opportunity to surpass the performance and energy efficiency of unstructured algorithms.

To leverage the potential of SSFT-Alg and address the inefficiencies of existing hardware in handling high sparsity, we propose SSFT-Hw, an accelerator designed with highly parallel dataflow and sparsity-aware modules that bypass unnecessary gradient computations and flexibly handle various stages of the fine-tuning process. In addition, SSFT-Hw can process Softmax and layer normalization operations, which are not supported by general SpM*SpM accelerators. This hardware is the first accelerator that can support sparse fine-tuning on LLMs, providing higher efficiency than GPUs and existing accelerators in handling high-sparsity fine-tuning. The main contributions of our work follow.

- We introduce a novel structured sparse fine-tuning algo-

TABLE I
DETAILED PROCESSES FOR LLM FINE-TUNING. VARIABLES INCLUDE L (LOSS), X (INPUT ACTIVATIONS), Y (OUTPUT ACTIVATIONS), W (WEIGHTS), γ (LEARNING RATE), AND W^* (UPDATED WEIGHTS).

Forward	$X \times W = Y$
Backward	$\frac{\partial L}{\partial Y} \times W^T = \frac{\partial L}{\partial X}$
Weight Gradient	$X^T \times \frac{\partial L}{\partial Y} = \frac{\partial L}{\partial W}$
Weight Update	$W - \gamma * \frac{\partial L}{\partial W} = W^*$

rithm called SSFT-Alg. Based on the inherent structural characteristics in parameter matrices, SSFT-Alg selects the top-k important rows or columns in the parameter matrix based on the gradient during the sampling process, establishing a pattern that allows for uniform access and processing of gradients while maintaining accuracy.

- We propose the SSFT-Hw accelerator, which leverages the sparse masks resulting from SSFT-Alg with a dataflow providing high parallelism and sparsity-aware modules. This design enables SSFT-Hw to achieve high performance and a low memory footprint with the regularity provided by SSFT-Alg.
- SSFT-Alg is verified on both BERT and LLaMA 2, demonstrating accuracy comparable to that of current algorithms. SSFT-Hw achieves $1.32\times$ higher throughput and $1.48\times$ greater energy efficiency compared to the SOTA while maintaining comparable accuracy.

II. BACKGROUND AND MOTIVATION

A. Fine-tuning process of LLMs

Taking stochastic gradient descent, one of the standard fine-tuning algorithms for LLMs, as the example, it comprises four stages, illustrated in Table I:

- The forward stage runs the inference algorithm to determine the model's predictions for training inputs and calculate the loss L (i.e., the training error). This consists of convolving the input tensor X with a set of weights W to obtain the output tensor Y .
- The backward stage propagates the loss gradient across the model's layers. This is done by convolving the loss gradient with respect to the output gradients $\frac{\partial L}{\partial Y}$ with transposed weights W^T .
- The weight gradient generation stage calculates the gradients that are utilized in updating weights. This consists of convolving the backpropagated loss gradients with respect to the gradient $\frac{\partial L}{\partial Y}$ with the transposed input X^T .
- The weight update stage determines how much weight should be decreased by computing the multiplication of learning rate γ and gradients $\frac{\partial L}{\partial W}$.

B. Motivation

To further illustrate the suboptimal performance of the SFT algorithm on GPUs, we compare SFT with LoRA (set to rank 64 to achieve the same sparsity as SFT) in terms of the number of operations, latency, and efficiency (ops/cycle). As shown in Fig. 3, despite original SFT processes 41.6% fewer number of operations than LoRA, which has the potential to outperform LoRA on GPUs, its latency and efficiency

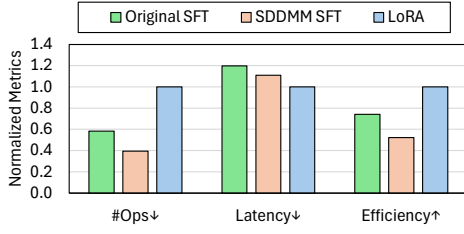


Fig. 3. Comparison of original SFT, SDDMM SFT, and LoRA in terms of number of operations, latency, and efficiency (ops/cycle). All metrics are normalized to LoRA. $\uparrow$ indicates that taller bars are better, while $\downarrow$ indicates the opposite. While the SDDMM version of SFT reduces 32.6% #Ops, it reduces latency by only 7.2% and decreases efficiency by 25.7%.

is 20.0% and 25.9% lower, respectively. Then we optimize the core operation, gradient generation, in SFT with a more efficient implementation called Sampled Dense-Dense Matrix Multiplication (SDDMM) [16], which is supported by the NVIDIA cuSPARSE library. Enhanced with SDDMM, GPUs calculate only the necessary weight gradients instead of the entire weight gradient matrix. Despite SDDMM reducing 32.6% operations, as shown in Fig. 3, total latency decreases less than only 7.2% over the original SFT, and the efficiency even becomes worse. This decline is due to low-computation-density operations such as gather_scatter and topk in SFT prevent optimal performance. Specifically, gather_scatter and topk account for 32.2% of cycles but only 0.8% of operations, which significantly harms efficiency. Therefore, based on current GPUs and optimization libraries like cuSPARSE, the potential of SFT cannot be fully realized. Both an efficient algorithm and a customized accelerator capable of efficiently processing SFT are critically required.

III. STRUCTURED SPARSE FINE-TUNING

One of the main contributions of this paper is a method for pre-computing a structured sparse subset of a model's parameters to update over many subsequent training iterations. To construct such a subset, we calculate the average of gradients as a signal of how important the parameter is for a given task during sampling. We then select the top-k rows and columns from the gradient matrix to create a subset of it. We refer to the resulting subset (i.e., a list indicating which parameters are included in the subset) as a *mask*. In this section, we detail the steps necessary for computing a structured sparsity mask. This process is diagrammed in Fig. 4 and listed in Algorithm 1.

A. Crucial Parameter Detection

Unlike previous works that utilize complex and time-consuming methodologies such as Fisher information [21], lottery-ticket [2], and delta vectors [3], SSFT-Alg adopts a static top-k gradient selection method to reduce running overhead while maintaining accuracy. The first step of SSFT-Alg is to select the subset of parameters that are crucial to update. One way to generate this subset is to consider how much each of the parameters will be impacted by the training data. In the general training process of neural networks, gradients are utilized to update the parameters with different

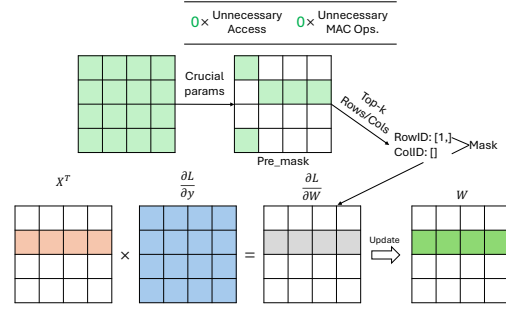


Fig. 4. An example of SSFT-Alg involves calculating trainable parameters of 4×4 weight matrix. SSFT-Alg selects the crucial parameters based on gradients after sampling. It selects rows or columns with the highest number of crucial parameters, incorporating them into a list of trainable elements. This list, comprising all trainable rows and columns, is employed during gradient computation to minimize unnecessary access and processing.

Algorithm 1 Structured Sparse Fine-Tuning Algorithm

```

1. Input:  $X, W_{original}, \text{sampling\_steps}, \text{sparsity}$ 
2. Output:  $W$ 
3. // Crucial Parameter Detection
4. for  $i = 1$  to  $\text{sampling\_steps}$  do
5.    $\text{loss} \leftarrow \text{Forward}(X[i], W)$ 
6.    $\text{grad\_X} \leftarrow \text{Backward}(W, \text{loss}, \text{grad\_Y})$ 
7.    $\text{grad\_W} \leftarrow \text{Weight\_Grad}(X[i], W, \text{grad\_Y})$ 
8.    $W \leftarrow \text{Weight\_Update}(W, \text{grad\_W})$ 
9.    $\text{grad\_mask} \leftarrow \text{grad\_mask} + \text{grad\_W}$ 
10. end for
11.  $W \leftarrow W_{original}$ 
12.  $\text{grad\_mask} \leftarrow \text{grad\_mask} / \text{sampling\_steps}$ 
13. for all  $l$  in layers do
14.    $\text{pre\_mask}[l] \leftarrow \text{top-k}(\text{grad\_mask}[l], \text{sparsity})$ 
15. end for
16. // Mask Generation
17. for all  $mkl$  in  $\text{pre\_mask}$  do
18.   for all  $\text{row}$  in  $mkl$  do
19.      $\text{row\_grad}[\text{row}] \leftarrow \text{count\_nonzero}(\text{row})$ 
20.   end for
21.   for all  $\text{col}$  in  $mkl$  do
22.      $\text{col\_grad}[\text{col}] \leftarrow \text{count\_nonzero}(\text{col})$ 
23.   end for
24.    $\text{mask}[mkl] \leftarrow \text{top-k}([\text{row\_grad}, \text{col\_grad}], \text{sparsity})$ 
25. end for
26. // Weight Gradient Generation and Weight Update
27. for  $\text{iter} = 1$  to  $\text{training\_steps}$  do
28.    $\text{loss} \leftarrow \text{Forward}(X[i], W)$ 
29.    $\text{grad\_X} \leftarrow \text{Backward}(W, \text{loss}, \text{grad\_Y})$ 
30.    $\text{grad\_W} \leftarrow \text{Weight\_Grad}(X[i], W, \text{grad\_Y}, \text{mask})$ 
31.    $W \leftarrow \text{Weight\_Update}(W, \text{grad\_W}, \text{mask})$ 
32. end for
33. return  $W$ 

```

types of optimizers. As lines 4-9 show, SSFT-Alg takes the *sampling_steps* as a parameter to decide the number of steps required by sampling. The parameters are updated in the same way as done in the general training process. The gradients of each of the parameters are then accumulated. After collecting all gradients, weights are reset to the original weights to avoid the impact of the sampling step. After completing the sampling process, the gradient matrices of each layer are scanned. The top-k gradients are selected from the matrices based on the

sparsity requirement from the input. These generated masks are named *pre-masks*. Despite current SFT algorithms that can generate or detect crucial parameters, they leave irregularity and dynamic detection overhead that harm both performance and memory access. In the next section, we describe how SSFT-Alg takes this data to introduce structured sparsity.

B. Mask Generation

Recall that our goal is to get subsets of trainable parameters in each layer with structured sparsity to update over many iterations of training while keeping the remainder of the parameters fixed. SSFT-Alg scans all pre-masks of each layer horizontally and vertically. During the scan process, SSFT-Alg counts non-zero values of each row and column (lines 17-25). After scanning all pre-masks, SSFT-Alg selects rows or columns with the top-k number of non-zero values as the final masks. Fig. 2 shows an example of this step. Intuitively, the pre-mask with irregular distribution is converted to a structured distribution with more regularity. Taking Fig. 4 as an example, there are 5 parameters selected as the crucial parameters. Two of these values are in the first column and three of these values are in the second column. If k is set to 1, which means 1 row or column with the maximal number of values is selected, the second row is selected as the *mask*. The mask generation stage of SSFT-Alg can be utilized not only with our crucial parameter detection method but also can cooperate with any of the mask selection methodologies as a plug-in, like Fish [21], Delta [3], etc.

C. Weight Update

After generating the structured sparse masks, these masks are utilized in the weight-updating stage. As shown at the bottom of Fig. 4, the gradients are computed based on the generated masks. Specifically, only rows and columns recorded in the list are required to be computed in the weight-updating stage. Therefore, SSFT-Alg only needs to access specific rows or columns of input matrices, which are X^T and $\frac{\partial L}{\partial Y}$, to generate necessary gradients ($\frac{\partial L}{\partial W}$). Furthermore, to update the weight matrix with the gradients of each weight, only specific rows and columns are updated.

IV. MICROARCHITECTURE

Fig. 5(a) shows an overview of SSFT-Hw. Two critical sparsity-aware modules, Memory Fetcher and Sparse-Dense Scheduler, are highlighted for clearer visualization of their details. Data is brought to the on-chip buffers from DRAM by the direct memory access (DMA) controller. Memory Fetcher fetches distinct types of data according to the stages and transfers data to the on-chip buffer. It also sends information about mask and sparse-dense mode to the Sparse-Dense Scheduler. The buffers store the activations, gradients, and weights, respectively. Sparse-Dense Scheduler passes data from on-chip buffers to the PE array. The PE array uses the compressed data and compression information in the *mask* to perform the main compute operations in the LLM fine-tuning.

A. Sparsity-aware Design

Due to irregularity and limited sparsity support, existing hardware platforms struggle with sparse fine-tuning. To efficiently support the structured sparse fine-tuning algorithms, two sparsity-aware modules, Memory Fetcher (Fetcher) and Sparse-Dense Scheduler (Scheduler), are introduced into the SSFT-Hw. Combined with a novel structured sparse dataflow shown in Fig. 5(b), Fetcher and Scheduler enable SSFT-Hw to process SSFT-Alg with high efficiency. There are four steps in fine-tuning LLMs: forward, backward, weight gradient generation, and weight update. SSFT-Hw processes weight gradient generation and weight update with sparse mode and forward and backward with dense mode.

In the sparse mode, to address the irregularity and sparsity issues shown in Fig. 1, Fetcher first takes the *mask* generated by Algorithm 1 as the additional input, along with transposed input X and gradient $\frac{\partial L}{\partial Y}$ from previous layer in the weight gradient generation stage. In *mask*, gradients to be updated are represented in a structured way, which stores only row and column indices to avoid the overhead of unstructured data storage in a general compressed format. Fetcher parses data from DRAM based on the information in *mask*. Parsed values are stored separately in on-chip buffers. *Mask* is then passed to the Scheduler for further processing. Scheduler reads data stored in on-chip buffers according to the information in *mask*, assigning data to different PEs based on row/column IDs for parallel processing, as shown at the bottom of Fig. 5(a). SSFT-Hw generates output matrix row by row or column by column following the information in *mask*. This also means that only several rows or columns of the input matrix (matrices X^T and $\frac{\partial L}{\partial Y}$) are required during the computation. After completing the computation of gradients, Fetcher fetches the relevant weights and updates weights with the structurally masked gradients without any unnecessary computation and memory access. In dense mode, SSFT-Hw performs dense-dense matrix multiplication during forward and backward stages in a tiled approach, a technique widely used in current accelerators [24].

The main modules present inside a PE are shown on the right of Fig. 5(a), which is the basic compute block in our accelerator. The PE design is based on an open-source design [24]. The data are stored in local registers of the PE by the input first-in-first-out (FIFO), weight FIFO, and gradient FIFO registers. PE takes the structured data and data information from DRAM. The PE then forwards data and information to the MAC lanes (for matrix multiplication), SoftMax modules (for SoftMax operation), or the Layer-Norm module (for layer normalization operation). MAC lanes and SoftMax modules can be utilized simultaneously with the schedule of the Scheduler to achieve high utilization. Finally, PE will write the results into input FIFO and then write the results back to the DRAM.

B. Dataflow

As shown in Fig. 5(b), a novel dataflow is proposed to enable high parallelism in SSFT-Alg. An $M \times N$ matrix

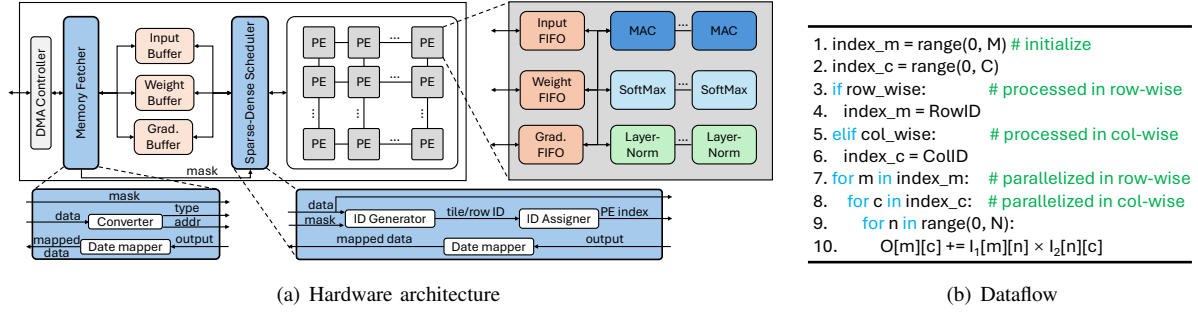


Fig. 5. (a) The hardware architecture of SSFT-Hw consists of the DMA controller, Memory Fetcher, on-chip buffers, Sparse-Dense Scheduler, and PE array. Each PE contains three types of FIFOs to store inputs, weights, and gradients. These FIFOs are connected with three types of computing units: MAC units, SoftMax units, and layer normalization units. (b) The dataflow used in SSFT-Hw to process the sparse weight gradient generation. Row or column is processed by a single PE in parallel.

TABLE II
SPECIFICATION OF SSFT-Hw.

Module	Configuration	Module	Configuration
DRAM Bandwidth	256GB/s	LayerNorm Units	1/PE
PEs	512	Act. Buffer	16MB
MAC Units	32/PE	Weight Buffer	64MB
SoftMax Units	32/PE	Grad. Buffer	16MB

(I_1) is multiplied by an $N \times C$ matrix (I_2) to generate an $M \times C$ matrix (O). Rows or columns to be processed are specified by array RowID or ColID (line 3 to line 6), which store mask information as shown in Fig. 4. Each row or column is processed by a dedicated PE in parallel, maximizing parallelism (lines 7 and 8). Consequently, each PE outputs a complete row or column (lines 9 and 10). This dataflow maximizes resource utilization, achieving high efficiency by eliminating zero values and irregularity.

V. EVALUATION

A. Experiment Setup

1) *Algorithm*: We demonstrate that our approach matches the performance of standard gradient-based training on the GLUE benchmark [26] while updating only 5% of the model’s parameters per task. All experiments for GLUE are run with the BERT_{LARGE} variant of BERT [8]. Sampling_step is 5.

To demonstrate the effectiveness of SSFT-Alg on larger models, we instruction-tune LLaMA 2 7B and 13B on Flan v2 [14] and GPT4-Alpaca [17] with 2.3% density mask. We evaluate instruction-tuned LLMs on the following benchmarks to capture a range of abilities: MMLU [10], TyDiQA [6], and HumanEval [5]. Furthermore, we compare SSFT-Alg with LoRA [11], as it offers the best performance–efficiency trade-off and is arguably the most widespread pre-existing PEFT method [7], as well as IA³ [13]. We use the LoRA and IA³ implementations in the PEFT library [15]. Experiments of BERT and LLaMA 2 7B are run on an NVIDIA A100-40GB and LLaMA 2 13B is run on an NVIDIA H100-96GB.

2) *Hardware*: To have a fair comparison with the state-of-the-art work, TransCODE, we set our hardware configuration the same. Table II shows the associated design choices. The

TABLE III
GLUE TEST SERVER EVALUATION RESULTS WITH BERT_{LARGE}.
TRAINING WITH THE SSFT-ALG GAINS COMPETITIVE ACCURACY WHILE
UPDATING ONLY 5% OF MODEL PARAMETERS.

Method	FullFT	Bit Fit	Diff Pruning	Monarch	Fish Mask	SSFT-Alg
QNLI	93.4	90.4	91.9	87.0	93.3	93.1
SST-2	94.0	94.5	93.8	92.4	93.6	93.1
MNLIimm	87.0	84.8	85.5	82.3	85.6	86.0
CoLA	66.0	60.3	61.0	59.6	64.8	64.1
MRPC	88.8	86.3	86.2	90.1	89.6	89.9
STS-B	90.6	85.0	85.6	88.3	90.6	90.1
RTE	73.6	69.6	67.5	75.0	75.4	75.4
QQP	91.2	78.5	80.1	87.7	89.1	88.6
AVG	85.6	81.2	81.5	82.8	85.2	85.0

RTL model is synthesized using Synopsys Design Compiler version P-2019.03, targeting the 14-nm FinFET technology library [9], which provides power and area information in the synthesis report. We fixed the clock rate to 700 MHz based on the delay of all modules in the proposed architecture. We set the number of multipliers to 16 in each PE. We run the simulation to get the latency based on the open-source simulator proposed by TransCODE [24] along with our hardware modules. We compare the performance of SSFT-Hw with many previously proposed accelerators.

B. Accuracy of SSFT-Alg

1) *BERT*: Our results on SSFT-Alg can be seen in Table III. SSFT-Alg training results in effectively the competitive accuracy (85.0%) with SOTA SFT [21] (85.2%) and standard “dense” fine-tuning (85.6%), despite updating just 5% of BERT_{LARGE} parameters. Specifically, SSFT-Alg reduces maximally 0.7% accuracy in each task and achieves only 0.2% accuracy reduction compared with Fish Mask. SSFT-Alg training is competitive with all other parameter-efficient transfer learning approaches.

2) *LLaMA 2*: We present the main results of our experiments in Table IV. For LLaMA 2 7B, we find that SSFT-Alg outperforms LoRA and IA³ consistently across evaluation benchmarks and instruction tuning datasets, including Flan v2 and GPT4-Alpaca. SSFT-Alg achieves better accuracy than LoRA on 5 out of 6 tasks. We also find that 4-bit quantization

TABLE IV
ACCURACY OF PEFT AND QUANTIZED PEFT ALGORITHMS ON A RANGE OF EVALUATION TASKS FOR VARIOUS INSTRUCTION TUNING DATASETS.

Method	Flan v2 MMLU				Flan v2 TyDiQA				GPT4-Alpaca HumanEval			
	LLaMA 2 7B		LLaMA 2 13B		LLaMA 2 7B		LLaMA 2 13B		LLaMA 2 7B		LLaMA 2 13B	
	Original	Quant	Original	Quant	Original	Quant	Original	Quant	Original	Quant	Original	Quant
Original	45.8	44.8	55.3	54.7	50.9	50.2	60.3	59.6	13.5	11	17.8	15.5
FullFT	50.5	-	-	-	55.5	-	-	-	15.2	-	-	-
IA ³ [13]	46.7	-	55.1	-	51.6	-	60.1	-	14.7	-	18.5	-
LoRA [11]	49.3	47.7	55.8	55.0	55.3	54.3	61.4	60.7	15.7	13.3	19.8	18.2
SpIEL [3]	50.4	48.8	56.1	55.4	56.3	54.9	62.6	61.1	15.6	15.3	20.1	18.5
SSFT-Alg	50.2	48.8	56.3	55.2	56.2	54.8	62.5	61.2	15.6	14.3	20.5	18.6

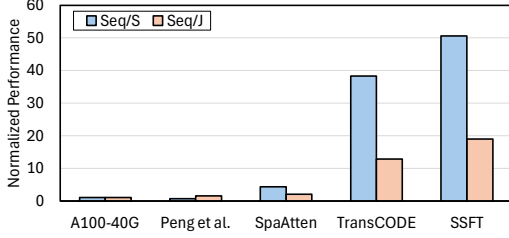


Fig. 6. Performance and energy efficiency comparisons. Seq/S represents sequences per second, and Seq/J represents sequences per joule. Implementation on A100-40GB is based on original BERT_{base} [8].

TABLE V
LATENCY AND ENERGY EFFICIENCY OF RUNNING LARGER MODELS ON DIFFERENT PLATFORMS. TRANS CODE CANNOT SUPPORT LLaMA 2 7B DUE TO THE OUT-OF-MEMORY (OOM) ISSUE.

Model	Design	Seq/S	Seq/J
BERT _{LARGE}	A100	31.90	0.13
	TransCODE	357.90	2.06
	SSFT-Hw	476.61	3.13
LLaMA 2 7B	A100	2.21	0.01
	H100	6.40	0.02
	TransCODE	OOM	OOM
	SSFT-Hw	8.70	0.04

results in only a modest reduction in performance across PEFT methods, and their relative performance remains similar. These results demonstrate that SSFT-Alg is a competitive PEFT method even in extremely resource-constrained scenarios. Meanwhile, SSFT-Alg can also achieve competitive accuracy with the SOTA SFT, SpIEL [3], with less than a 0.2% accuracy drop for all but one task. On LLaMA 2 13B, SSFT-Alg archives less than 0.2% accuracy drop. On some of the tasks, such as MMLU, SSFT-Alg even achieves better accuracy than SpIEL. Overall, SSFT-Alg is competitive in all 6 combinations of scales and instruction tuning datasets on LLaMA 2 with less than a 1% accuracy drop. Hence, SSFT-Alg appears to be a strong method that provides competitive accuracy with a hardware-friendly algorithm.

C. Performance and Energy Efficiency of SSFT

Fig. 6 shows performance comparisons of SSFT with various platforms. These platforms include NVIDIA A100, which is widely used in LLM fine-tuning. We also add co-design methods implemented on ASIC-based accelerators [24], [27] and FPGAs [18]. The throughput and energy values are normalized with respect to the A100 GPU. Since some of the

baselines do not support training, we report the performance values based on the inference performance scaling with the percentage of computation in the inference stage proposed in [24]. SSFT achieves $51.0\times$ higher throughput and $19.0\times$ lower energy consumption when compared to the off-the-shelf A100 GPU. SSFT also outperforms the state-of-the-art sparsity-aware Transformer training accelerator TransCODE $1.32\times$ on throughput and $1.48\times$ on energy consumption, respectively. Furthermore, to demonstrate the scalability of our methodology, we present the latency and energy consumption on BERT_{LARGE} and LLaMA 2 7B in Table V. On BERT_{LARGE}, SSFT-Hw outperforms A100 and TransCODE $14.94\times$ and $1.33\times$, respectively, in terms of latency (Seq/S). And it outperforms TransCODE $1.52\times$ on energy efficiency (Seq/J). On LLaMA 2 7B, TransCODE fails to support fine-tuning due to memory limitations. Consequently, SSFT-Hw is $3.9\times$ and $1.4\times$ faster than A100 and H100, respectively, while saving $4.0\times$ and $2.0\times$ energy compared to the A100 and H100. These performance gains can be attributed to the support of structured gradient generation.

VI. CONCLUSION

This work introduces Structured Sparse Fine-Tuning (SSFT), a significant advancement in fine-tuning LLMs that bridges the gap between algorithmic efficiency and hardware capability. SSFT-Alg shows that integrating structured sparsity into the fine-tuning process is possible. Complemented by the SSFT-Hw accelerator which is designed to efficiently utilize sparse gradients with sparsity-aware modules and a novel dataflow, our approach not only enhances computational efficiency and reduces memory usage but also demonstrates superior performance and energy efficiency compared to existing methods. Validated on prominent models like BERT, LLaMA 2 7B and 13B, SSFT outperforms GPUs and the state-of-the-art sparsity-aware transformer accelerators by $51.0\times$ and $1.32\times$ in throughput and $19.0\times$ and $1.48\times$ in energy efficiency, respectively, with less than 1% accuracy reduction.

VII. ACKNOWLEDGMENTS

We thank the anonymous reviewers. This work is supported by the NUS Artificial Intelligence Institute (NAII-SF-2024-004) and the Singapore Ministry of Education (T2EP20222-0026). This work is also supported by IAF, A*STAR, SOITEC, NXP, and the National University of Singapore under FD-fAbrICS: Joint Lab for FD-SOI Always-on Intelligent & Connected Systems (I2001E0053).

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] A. Ansell, E. Ponti, A. Korhonen, and I. Vulić, “Composable sparse fine-tuning for cross-lingual transfer,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 1778–1796.
- [3] A. Ansell, I. Vulić, H. Sterz, A. Korhonen, and E. M. Ponti, “Scaling sparse fine-tuning to large language models,” *arXiv preprint arXiv:2401.16405*, 2024.
- [4] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, “BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 1–9.
- [5] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [6] J. H. Clark, E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev, and J. Palomaki, “TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 454–470, 2020.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [9] A. Guler and N. K. Jha, “Hybrid monolithic 3-d ic floorplanner,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 10, pp. 1868–1880, 2018.
- [10] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
- [11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [12] V. Lialin, V. Deshpande, and A. Rumshisky, “Scaling down to scale up: A guide to parameter-efficient fine-tuning,” *arXiv preprint arXiv:2303.15647*, 2023.
- [13] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. A. Raffel, “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 1950–1965, 2022.
- [14] S. Longpre, L. Hou, T. Vu, A. Webson, H. W. Chung, Y. Tay, D. Zhou, Q. V. Le, B. Zoph, J. Wei, and A. Roberts, “The flan collection: Designing data and methods for effective instruction tuning,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 22 631–22 648.
- [15] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, “PEFT: State-of-the-art parameter-efficient fine-tuning methods,” <https://github.com/huggingface/peft>, 2022.
- [16] I. Nisa, A. Sukumaran-Rajam, S. E. Kurt, C. Hong, and P. Sadayappan, “Sampled dense matrix multiplication for high-performance machine learning,” in *2018 IEEE 25th International Conference on High Performance Computing*. IEEE, 2018, pp. 32–41.
- [17] B. Peng, C. Li, P. He, M. Galley, and J. Gao, “Instruction tuning with GPT-4,” *arXiv preprint arXiv:2304.03277*, 2023.
- [18] H. Peng, S. Huang, S. Chen, B. Li, T. Geng, A. Li, W. Jiang, W. Wen, J. Bi, H. Liu, and C. Ding, “A length adaptive algorithm-hardware co-design of transformer on fpga through sparse attention and dynamic pipelining,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 1135–1140.
- [19] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, “Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training,” in *2020 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 2020, pp. 58–70.
- [20] H. Shao, J. Lu, M. Wang, and Z. Wang, “An efficient training accelerator for transformers with hardware-algorithm co-optimization,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [21] Y.-L. Sung, V. Nair, and C. A. Raffel, “Training neural networks with fixed sparse masks,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 193–24 205, 2021.
- [22] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [23] S. Tuli and N. K. Jha, “Acceltran: A sparsity-aware accelerator for dynamic inference with transformers,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [24] —, “Transcode: Co-design of transformers and accelerators for efficient training and inference,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [25] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, “Well-read students learn better: On the importance of pre-training compact models,” *arXiv preprint arXiv:1908.08962*, 2019.
- [26] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355.
- [27] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 2021, pp. 97–110.
- [28] M. Zhou, W. Xu, J. Kang, and T. Rosing, “Transpim: A memory-based acceleration via software-hardware co-design for transformer,” in *2022 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 2022, pp. 1071–1085.